

P4/Stratum活用に向けた プログラマブルスイッチの実装・検討

2018年11月19日

NTTネットワークサービスシステム研究所

武井 勇樹

Copyright©2018 NTT corp. All Rights Reserved.

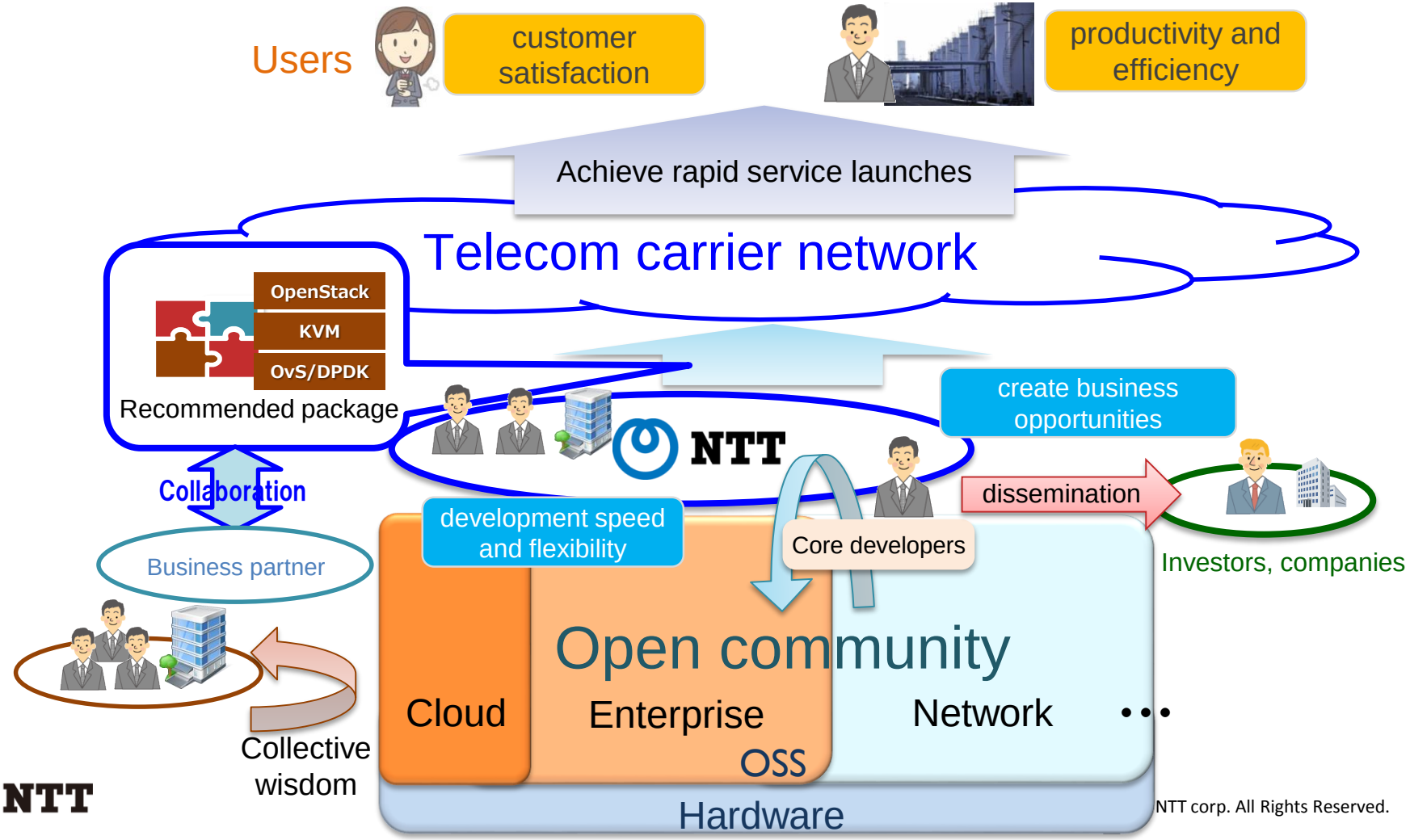
本日のアジェンダ

- 背景
- P4/Stratumの概要
- NTTのユースケース
- P4スイッチ/P4Runtimeの検討状況
- 今後の取り組み
- まとめ

Copyright©2018 NTT corp. All Rights Reserved.

キャリアネットワークにおけるOSS活用の推進

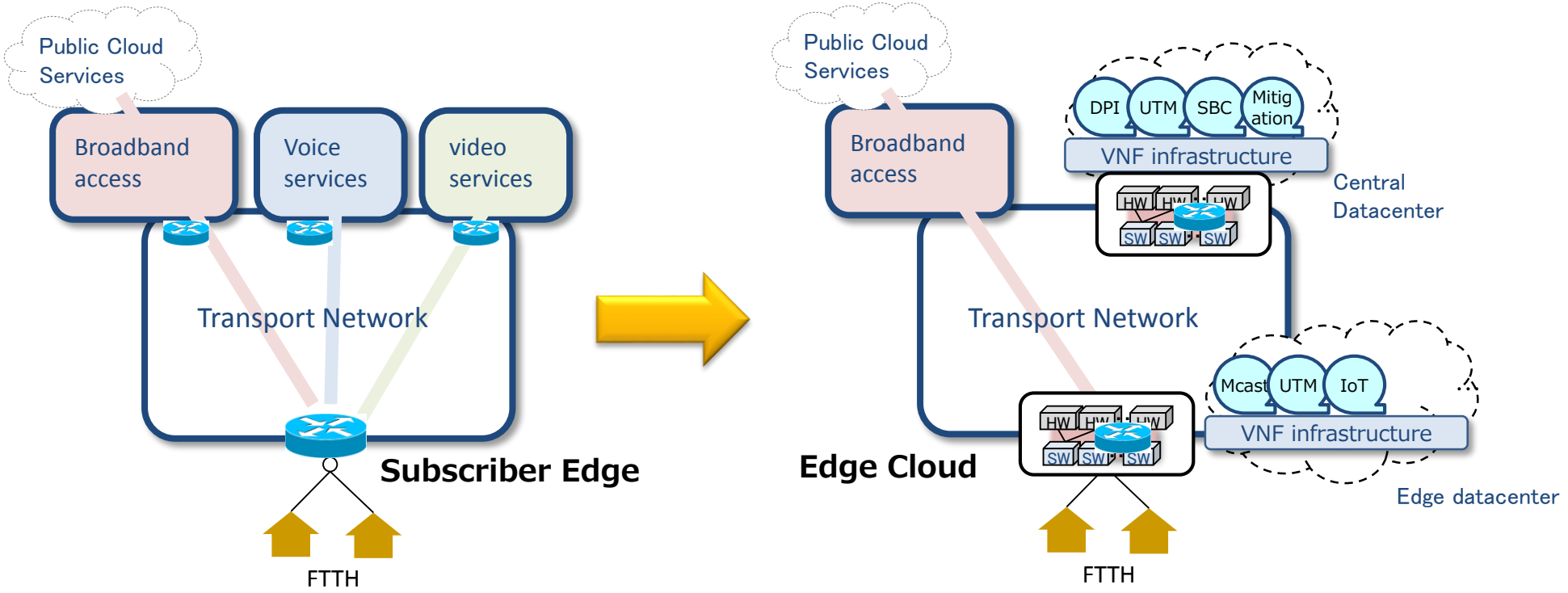
NTTは、「プロダクトのオープン化」と「OSSへの要件インプット」の両面からコミュニティ活動を実施し、通信キャリアネットワークへの適用を検討している。



NTT corp. All Rights Reserved.

将来ネットワークのイメージ

- 将来のネットワークにおいては、サービスの変更・拡張に対して、経済的かつ柔軟に対応するため、プログラマビリティが要求されると考えられる。
- この実現に向けて、P4/Stratumを活用できないか検討している。



- トリプルプレイサービス
- 加入者向けに最適化

- サービス変更/拡張にあわせ、ネットワーク機能を迅速に提供
- 新しい事業を始めたい事業者に応じたカスタマイズが可能

Copyright©2018 NTT corp. All Rights Reserved.

P4/Stratumの概要

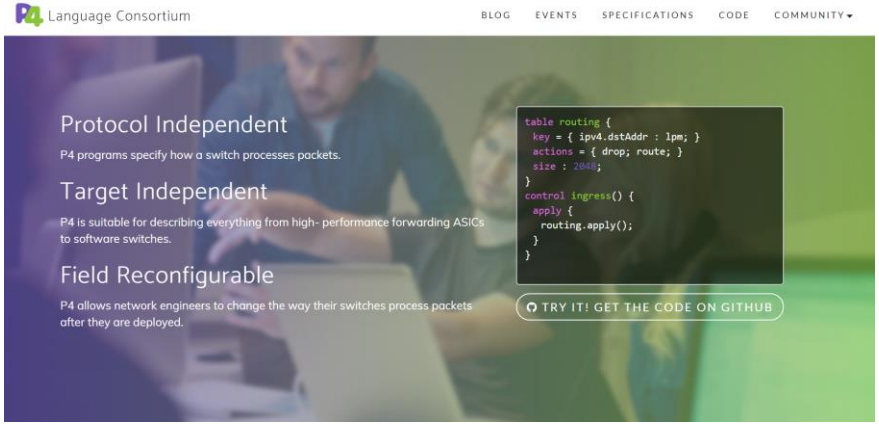
Copyright©2018 NTT corp. All Rights Reserved.

P4とは

- P4：
  High-level Language for
  Programming Protocol-independent Packet Processors　の略

- パケットを受け取ってから、その中身に応じて処理を行い送り出すまで(データプレーン)の動作を記述するためのプログラミング言語

- 『特定のプロトコルに依存せず，
  パケット処理の独自仕様を実現できる』
  (Protocol-independent)
- 『多様なハードウェアの仕様を意識せず，
  パケットの処理の内容を実装できる』
  (Target Independent)



https://p4.org/より引用

Copyright©2018 NTT Corp. All Rights Reserved.

【参考】 P4対応デバイス

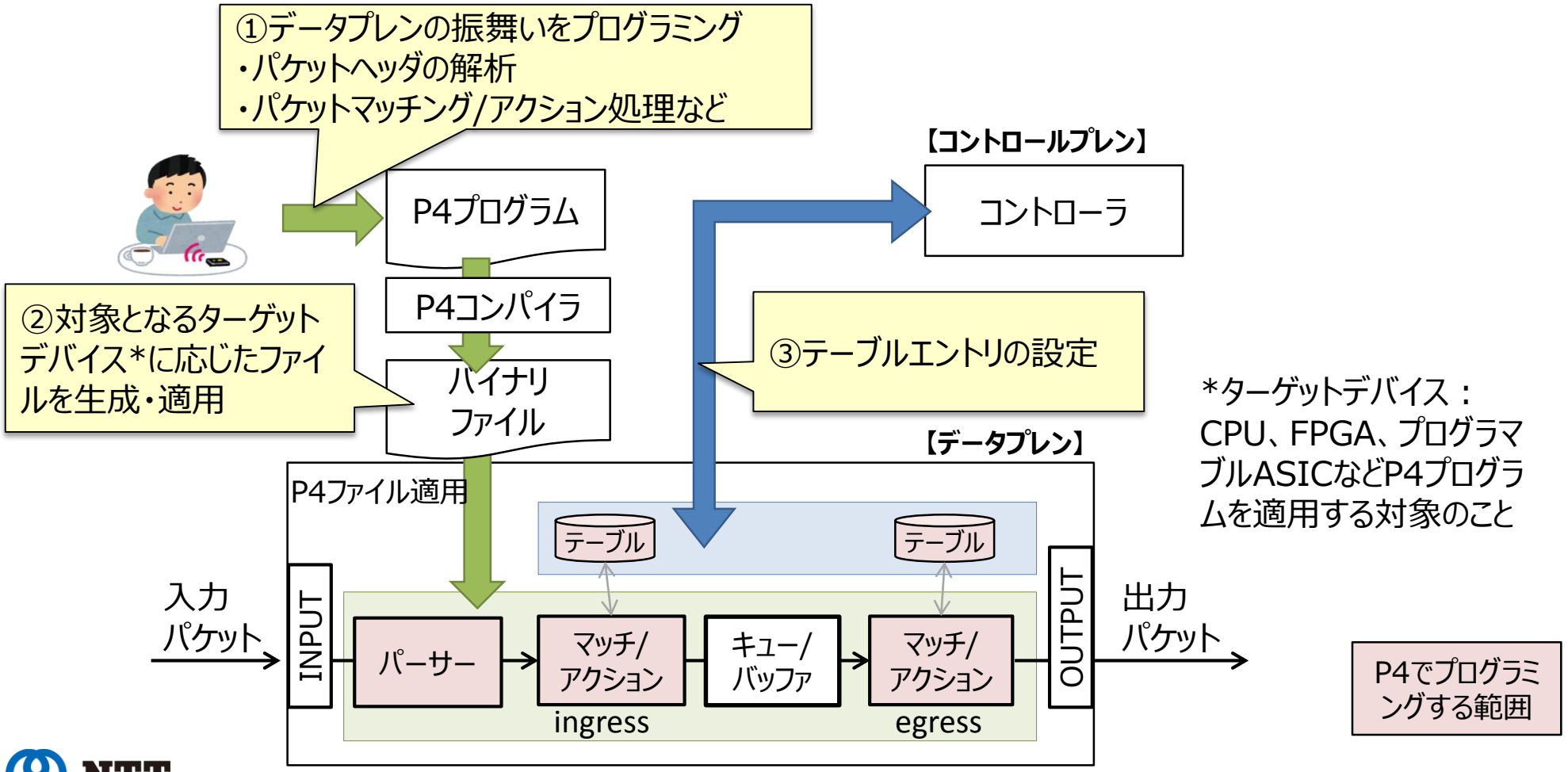
- CPU、FPGA、NPU、スイッチASICといった多様なデバイス種別に対応している。
- P4には現在２つの仕様バージョン($P4_{14}$ / $P4_{16}$)が存在し、どちらもメンテナンスが継続。

デバイス種別		フレームワーク例	コミュニティ/ベンダ例	サポートP4バージョン
CPU		BMv2	P4.org	P4_14/16
Smart NIC	FPGA	SDNet	Xilinx	P4_16
		NP4	Netcope	P4_14
	NPU	Agilio	Netronome	P4_14/16
Switch-ASIC		Tofino	Barefoot	P4_14/16

Copyright©2018 NTT Corp. All Rights Reserved.

P４でパケット転送するまでの基本的な流れ

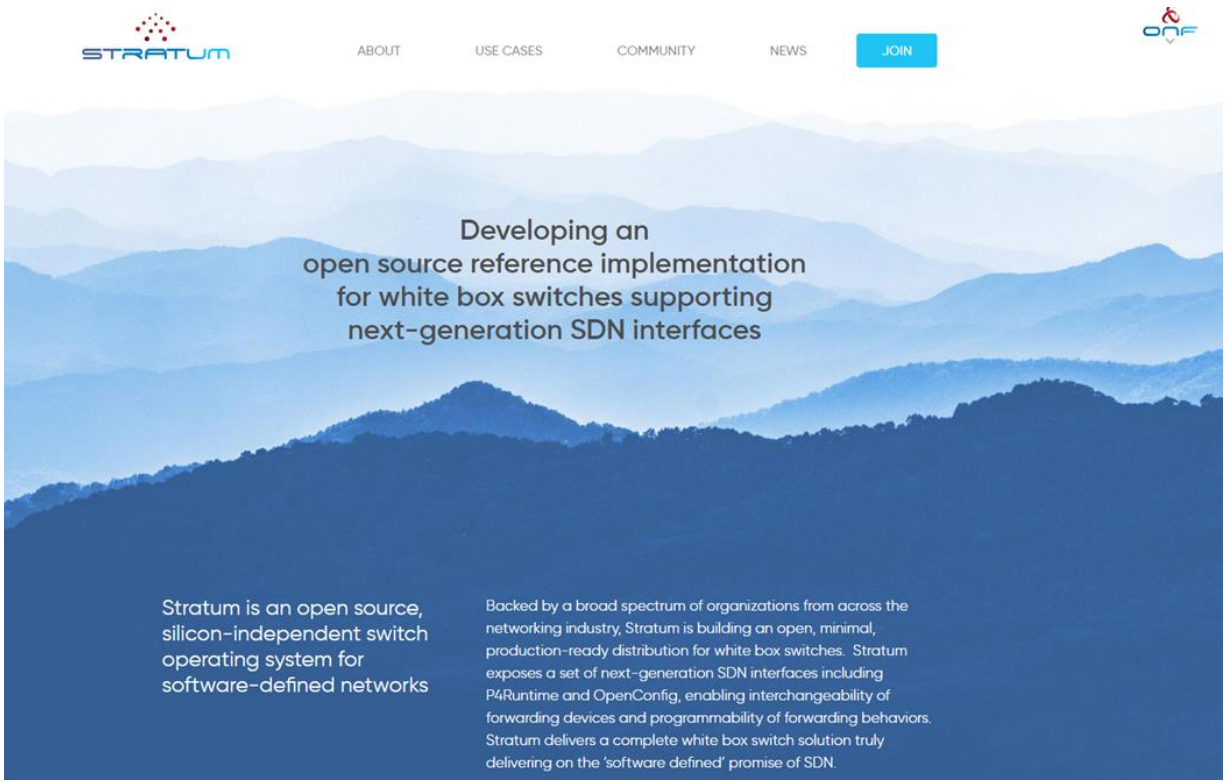
- データプレンの振舞いを定義するプログラム（P4プログラム）を準備し、コンパイル（①②）
- パケット転送に必要なテーブルエントリを設定し、パケットを転送（③）

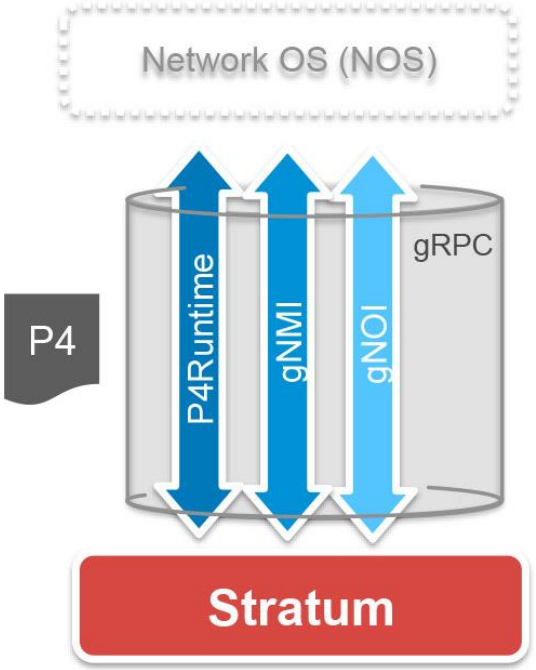


Copyright©2018 NTT corp. All Rights Reserved.

Stratumプロジェクトとは？

- ネットワークスイッチをソフトウェアで定義するオープンソースソフトウェア
- Stratumは、Google社・ONFらが主導するプロジェクト





https://stratumproject.org/ より引用

Copyright©2018 NTT corp. All Rights Reserved.

Stratum Community

Innovative R&D by NTT

Cloud Providers

Telecom Operators

Networking Vendors

White Box ODM Vendors

Silicon Vendors

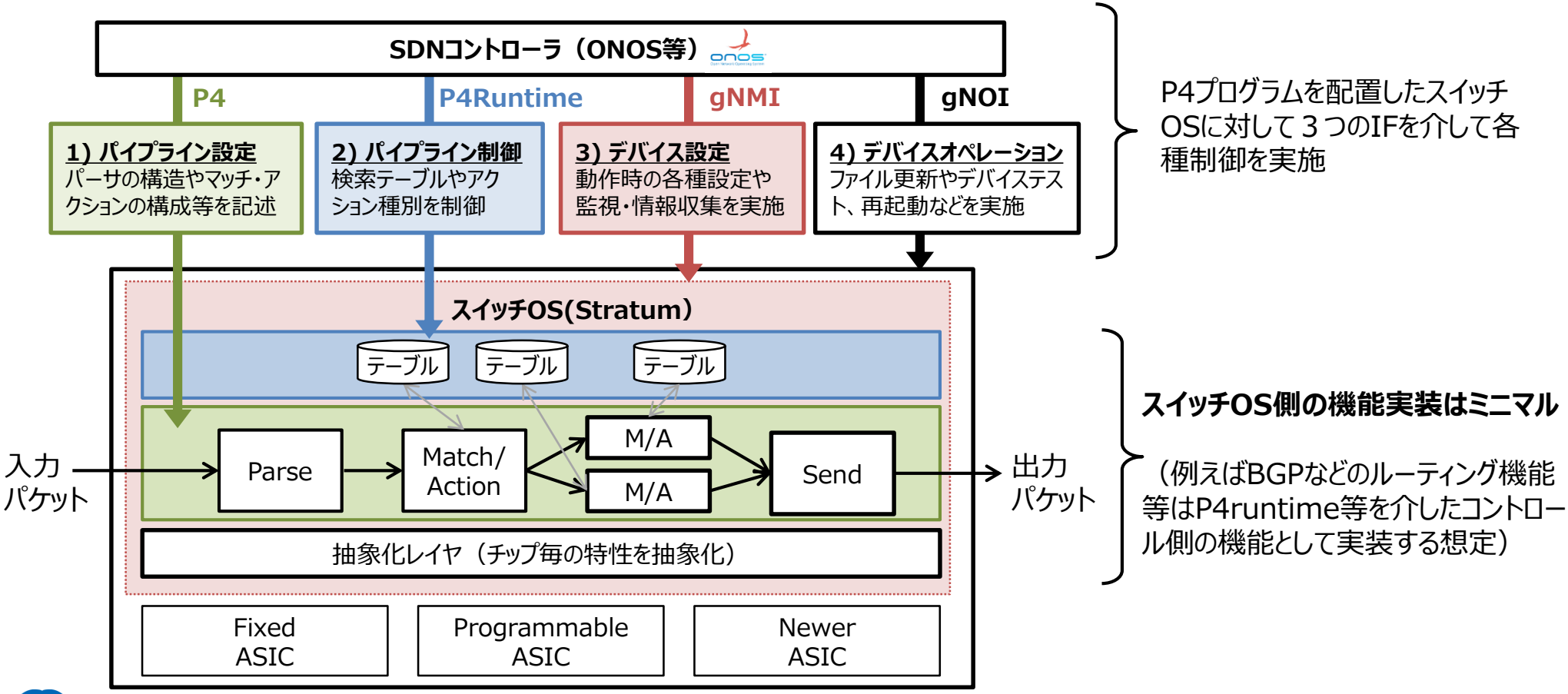
Other Open Source Projects

https://www.opennetworking.org/stratum/より

ights Reserved.

Stratum概要

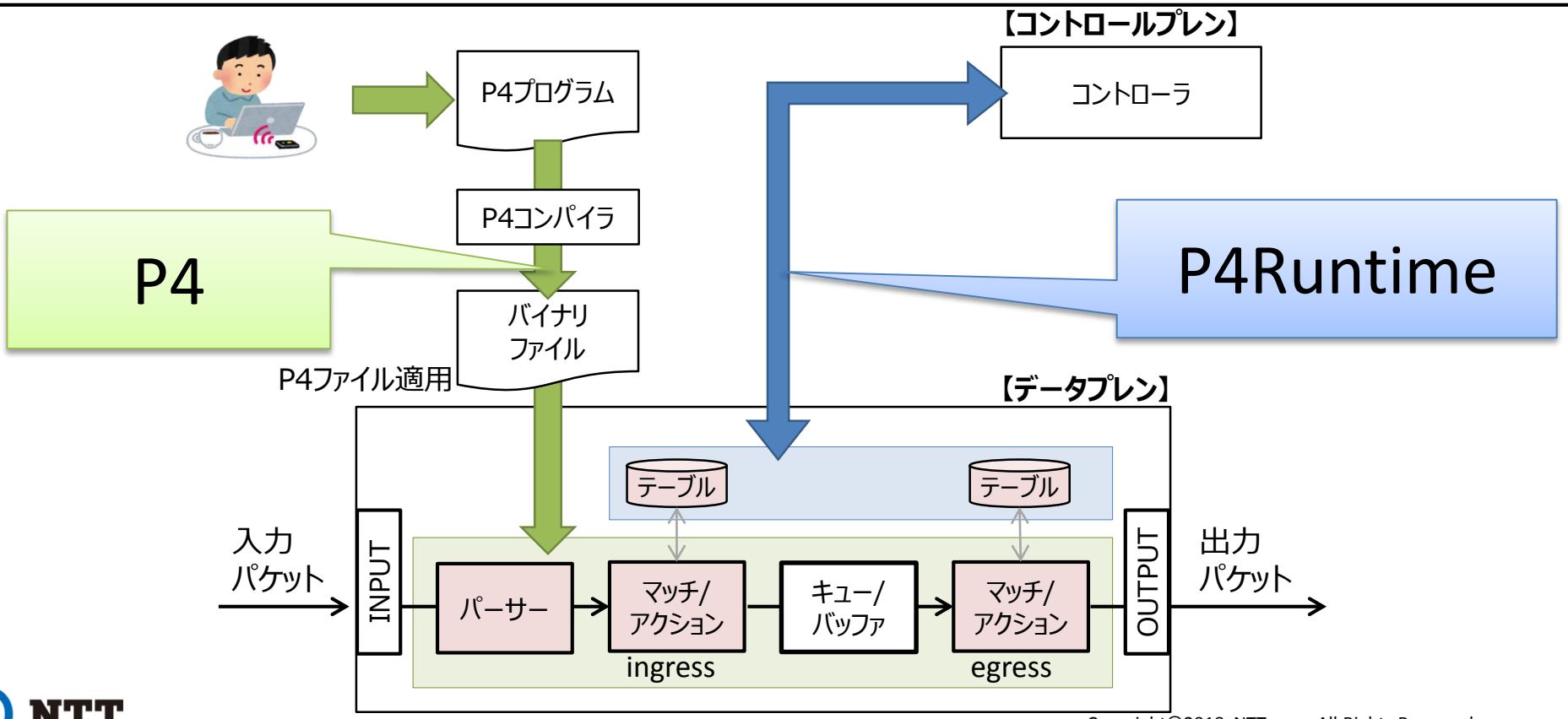
- SDN集中制御を前提とした、ミニマルな機能実装を指向したスイッチOSである。
- 異なるベンダチップを搭載したスイッチであっても統一的な手段で制御するために P4Runtime・gNMI・gNOIと呼ぶインタフェースを規定。



Copyright©2018 NTT corp. All Rights Reserved.

P4とP4 runtime

P4 ： デバイスを抽象化し、スイッチがどのようにパケットを処理するか定義。
　　パイプラインの指定(どのフィールドが一致するか、どんなアクションを実行するか等)。
P4Runtime ： 固定的な機能のスイッチであるか、プログラマブルであるかに関係なく、P4言語で動作が指定されている機能を制御するために使用するAPI。



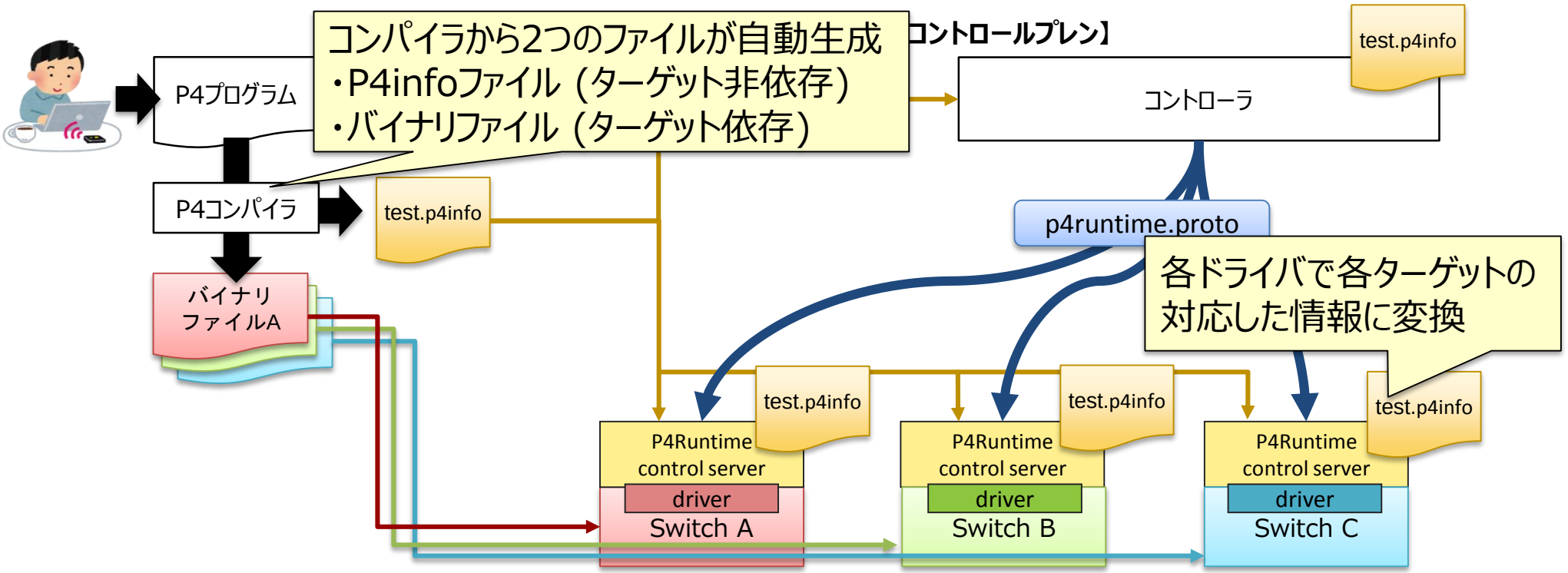
Copyright©2018 NTT corp. All Rights Reserved.

P4処理の流れについて

コンパイラから2つのファイルが自動生成。

①P4infoファイル（ターゲット非依存）：コントロールプレーンから制御するためのテーブルIDなどの対応関係が格納。外部コントローラはこのファイルを参照しながらターゲットデバイスを制御。

②バイナリファイル（ターゲット依存）：対象ターゲットデバイスに応じたデータプレーンファイル。



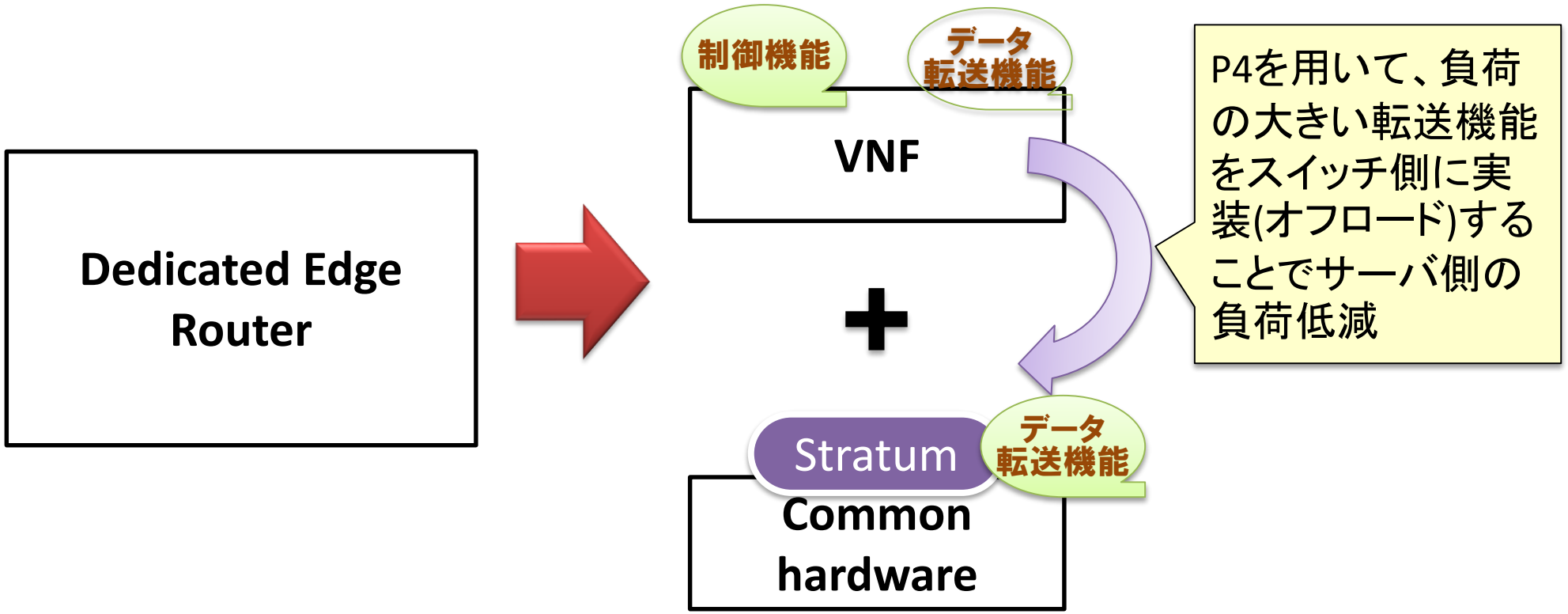
Copyright©2018 NTT Corp. All Rights Reserved.

NTTのユースケース

Copyright©2018 NTT Corp. All Rights Reserved.

ユースケース１　BNGのディスアグリゲーション

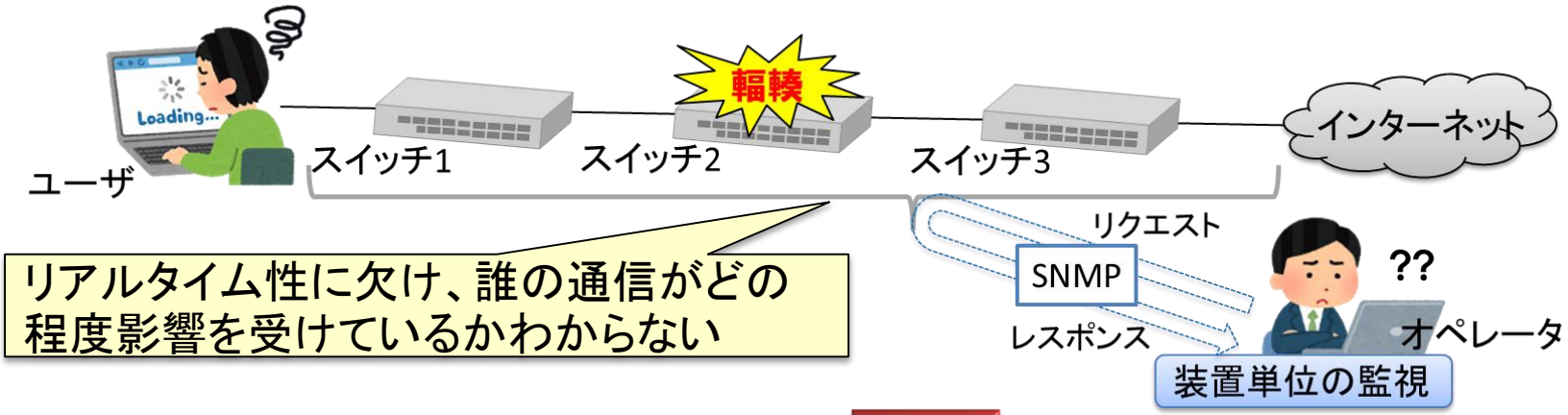
- ネットワークの柔軟性・スケール性の向上のため、既存の大型ルータを汎用装置とVNF技術による実現を目指している。



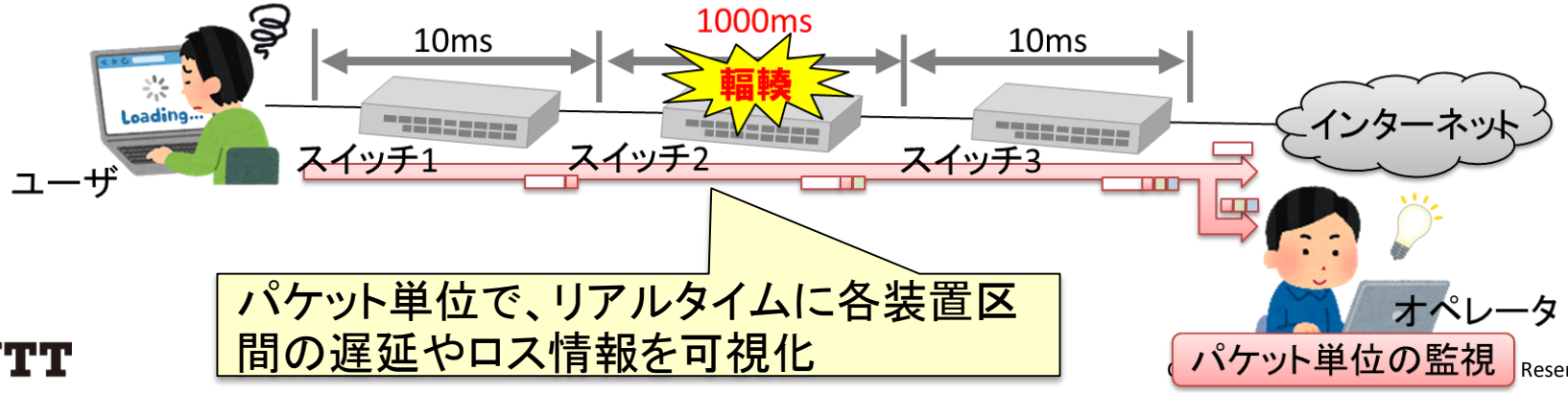
Copyright©2018 NTT corp. All Rights Reserved.

ユースケース2　通信品質の可視化

- 装置からパケット単位に情報取得ができ、通信品質を満たしているかを明確に可視化。
- パケット単位で情報を取得できるので、精緻な制御を実施可能なインフラを用意できる。



【 In-band Network Telemetry 】



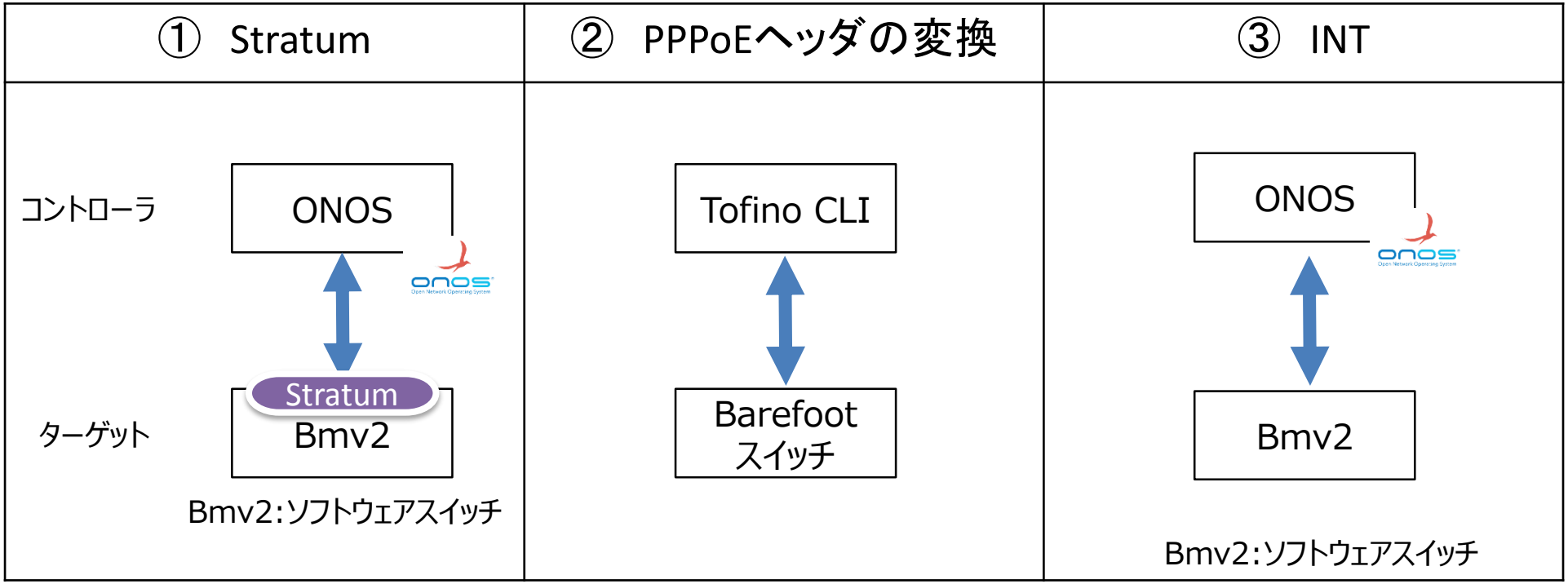
Reserved.

P4スイッチ/P4Runtimeの検討状況

Copyright©2018 NTT corp. All Rights Reserved.

取り組みの全体像

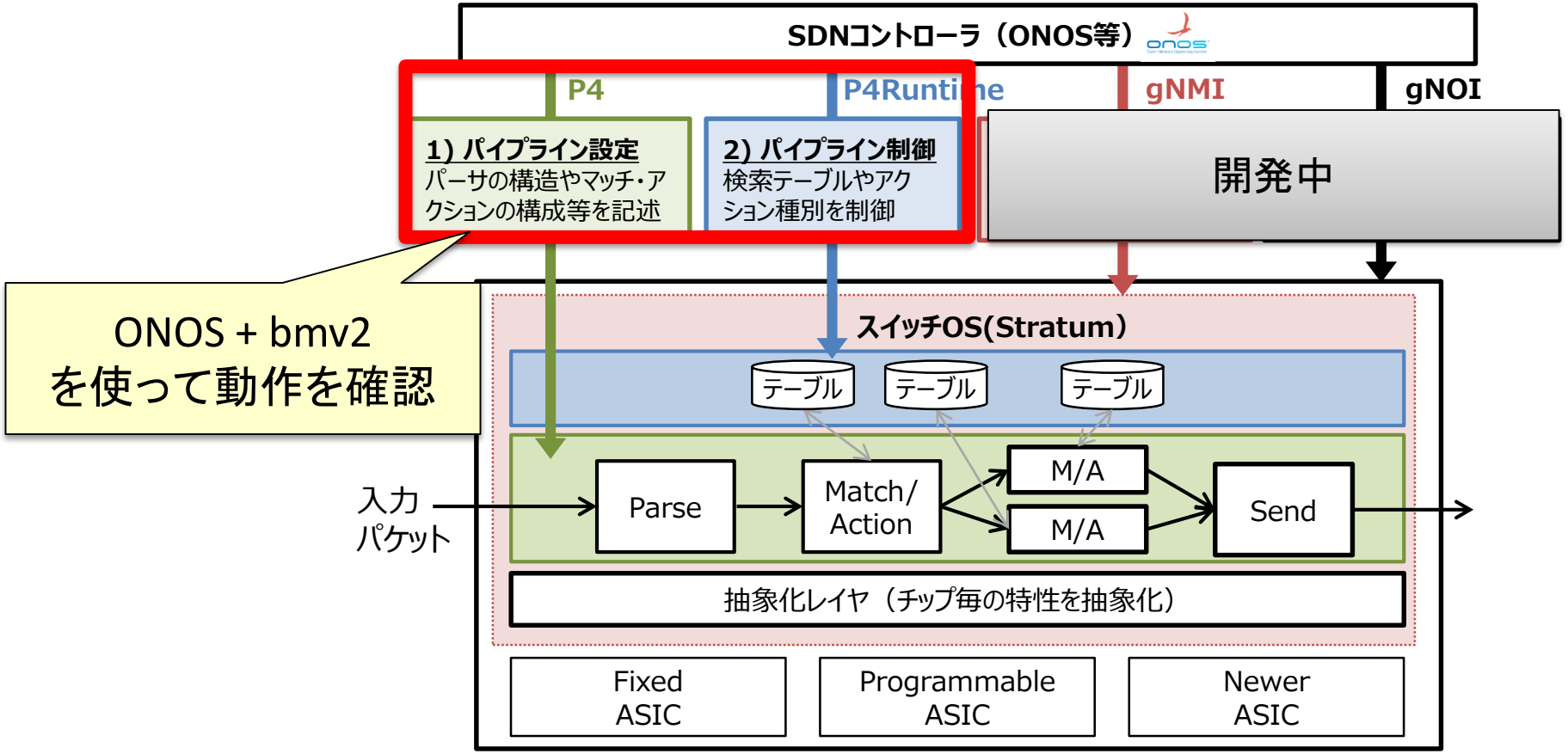
P4/Stratumを用いたユースケース実現に向けて、現在下記の評価・検討を実施。
① Stratumの実装状況の確認
②【ユースケース 1】P4によるハードウェアオフロードの実現性の確認 (PPPoEヘッダの変換)
③【ユースケース 2】P4によるINTの実現性の確認



Copyright©2018 NTT corp. All Rights Reserved.

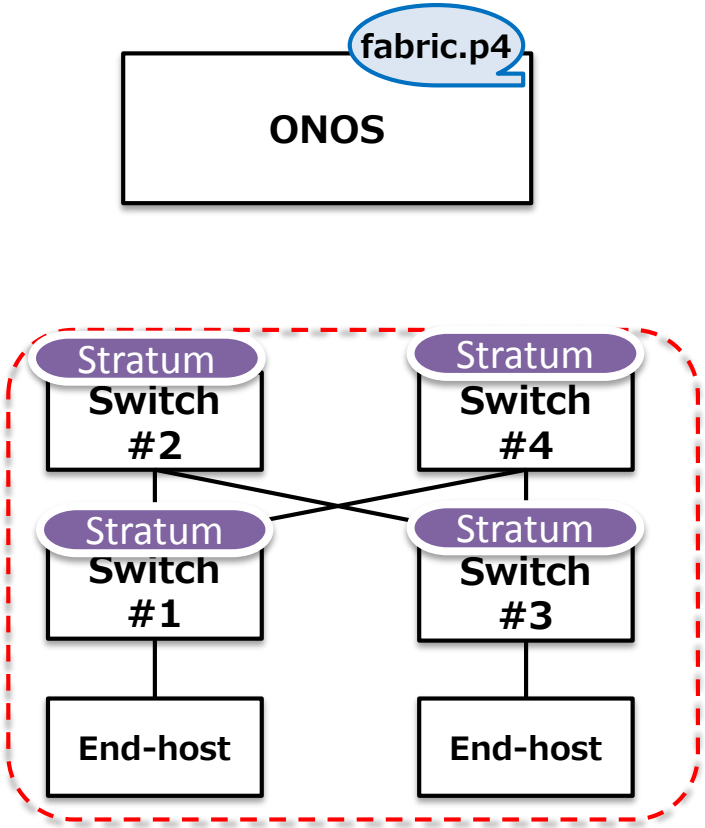
① Stratumの実装状況の確認 (1/2)

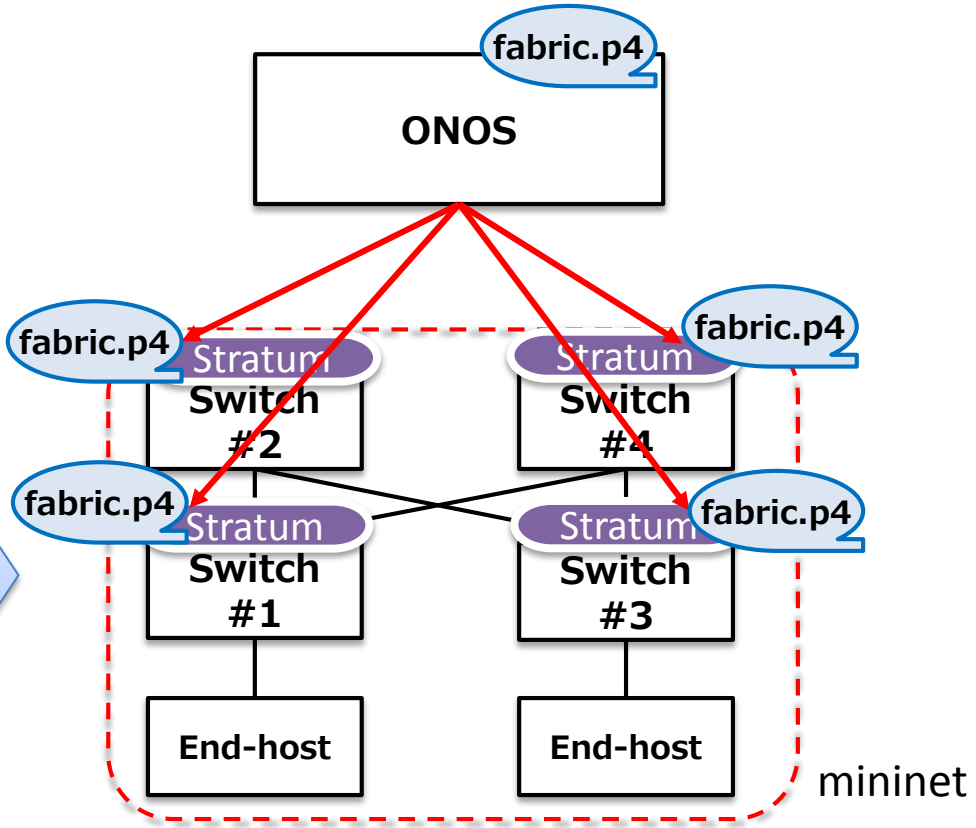
Stratumは現在も開発中
P4サービスは、bmv2向けに実装が完了しており、Stratumメンバー向けに展開がされている



Copyright©2018 NTT corp. All Rights Reserved.

① Stratumの実装状況の確認 (2/2)





Mininetを使って、
bmv2が4台のスイッチ
ファブリックを作成

P4ファイル(fabric.p4)
及びテーブル情報を
P4Runtimeで投入

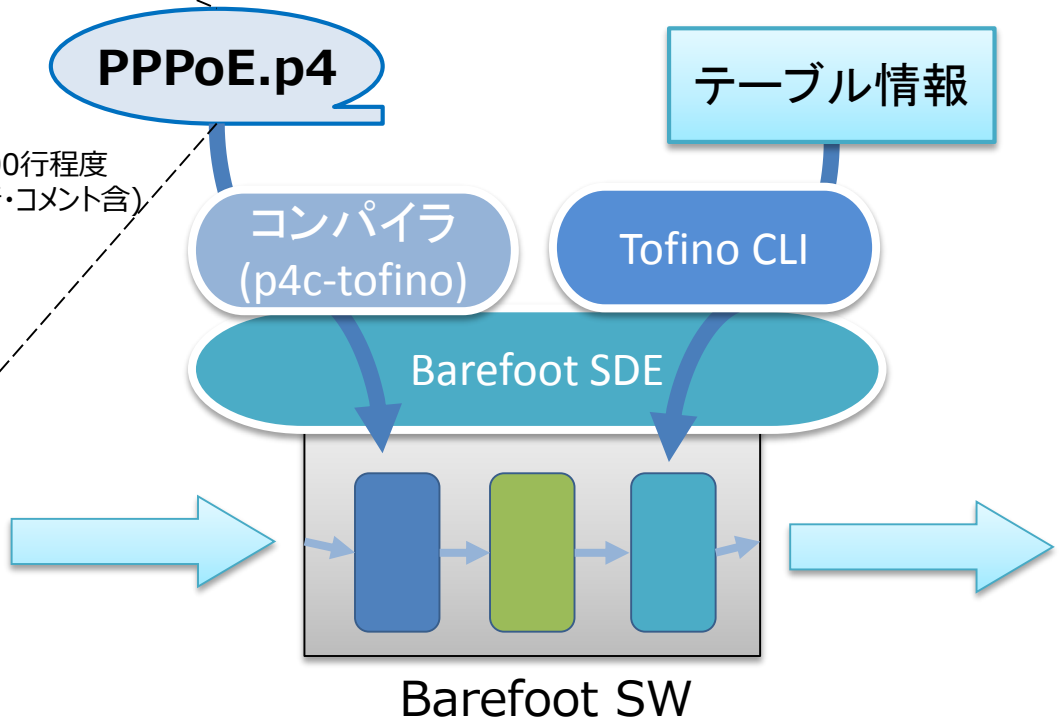
Copyright©2018 NTT corp. All Rights Reserved.

② P4によるPPPoEヘッダの変換の実装 （1/2）

- PPPoEパケットをL2TPパケットに変換するオリジナルコードをP4_14で作成。
  → 非常に簡単に実装が可能 (400行程度)
- Barefoot SDKを用いてスイッチ上に実装。

```
PPPoE.p4  (P4_14)
/********** headers **********/
…
header_type pppoe_t {
    fields {
        ver     : 4;
        pktType   : 4;
        code    : 8;
        sessionId     : 16;
        payloadLength : 16;
    }
}
…
/********** parser **********/
…
/********** action **********/
…
/********** table **********/
…
/********** control *
…
```

400行程度
(改行・コメント含)



Copyright©2018 NTT corp. All Rights Reserved.

② P4によるPPPoEヘッダの変換の実装 （2/2）

- 試験機より疑似パケットを印加し、実装した機能が動作するか確認。
- 下記のように、ハードウェアスイッチ上でパケットの変換動作が可能。




```
Frame 1: 974 bytes on wire (7792 bits), 974 bytes captured (7792 bits) on interface 0
Ethernet II, Src: MS-NLB-PhysServer-06_0a:0e:ff:f3 (02:06:0a:0e:ff:f3), Dst: Rivertre_0
802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 1001
PPP-over-Ethernet Session
    0001 .... = Version: 1
    .... 0001 = Type: 1
    Code: Session Data (0x00)
    Session ID: 0x0001
    Payload Length: 946
Point-to-Point Protocol
Internet Protocol Version 4, Src: 2.2.2.2, Dst: 1.1.1.1
User Datagram Protocol, Src Port: 60002, Dst Port: 60001
Data (916 bytes)
```

PPPoE

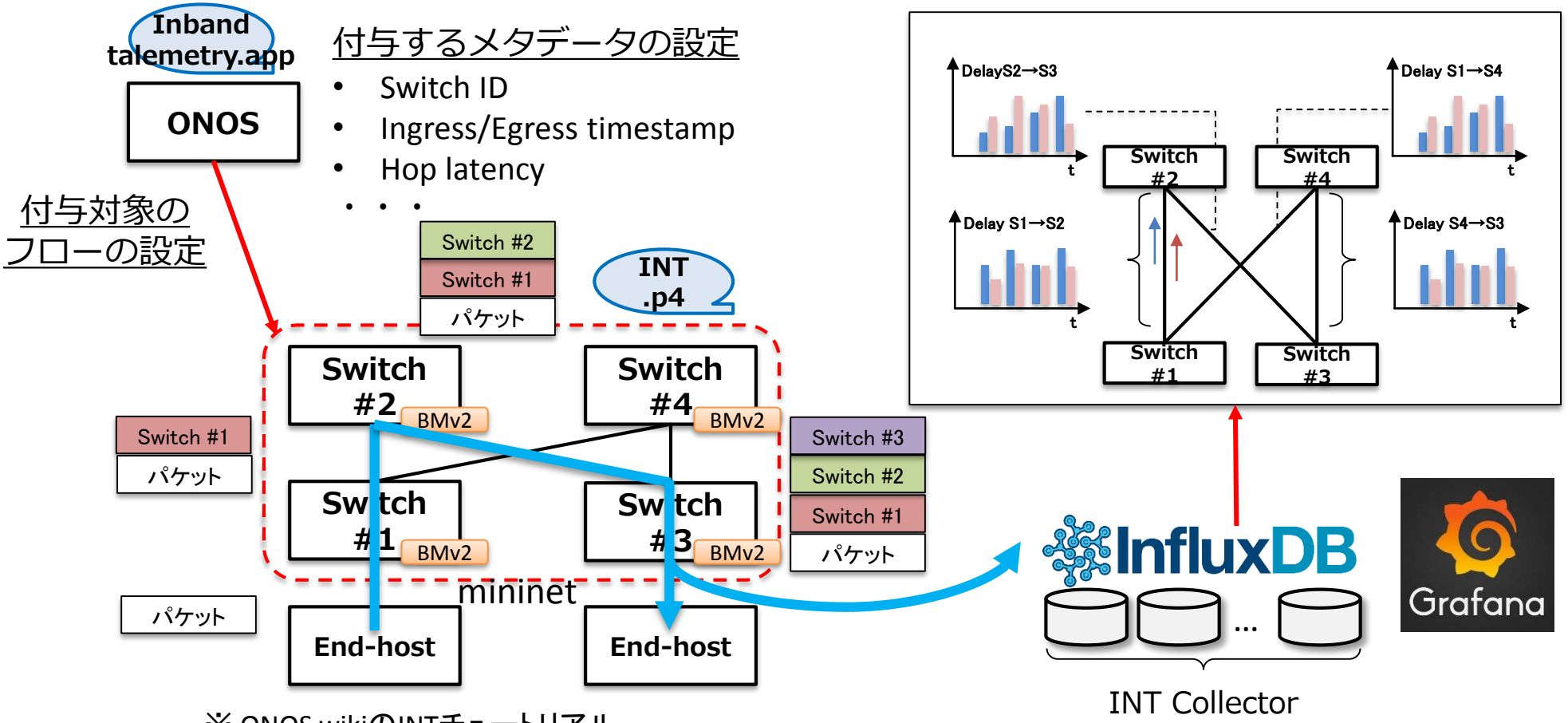
```
Frame 1: 1000 bytes on wire (8000 bits), 1000 bytes captured (8000 bits) on interface 0
Ethernet II, Src: Cisco_00:00:01 (00:12:01:00:00:01), Dst: AlaxalaN_e3:0b:bb (00:12:e2:e
Internet Protocol Version 4, Src: 10.3.0.2, Dst: 100.0.0.1
User Datagram Protocol, Src Port: 1701, Dst Port: 1701
Layer 2 Tunneling Protocol
  Packet Type: Data     Message Tunnel Id=61847 Session Id=64798
  Tunnel ID: 61847
  Session ID: 64798
Point-to-Point Protocol
Internet Protocol Version 4, Src: 2.2.2.2, Dst: 1.1.1.1
User Datagram Protocol, Src Port: 60002, Dst Port: 60001
Data (916 bytes)
```

L2TP

Copyright©2018 NTT corp. All Rights Reserved.

③ P4によるINTの実現性の確認 (1/3)

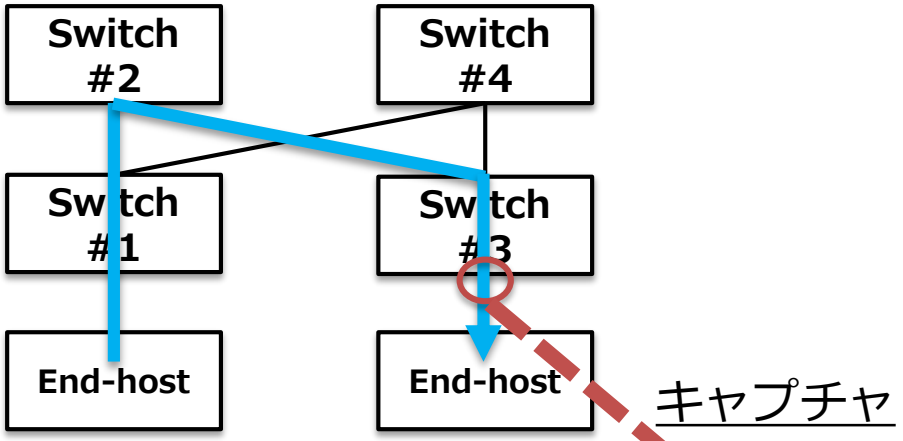
- INT(In-band Network Telemetry) とONOSを用いたINT実装を確認。
- ONOS wikiのINTチュートリアルを使用。



※ ONOS wikiのINTチュートリアル
https://wiki.onosproject.org/display/ONOS/%28INT%29+In+Band+Network+Telemetry+with+ONOS+and+P4

Copyright©2018 NTT corp. All Rights Reserved.

③ P4によるINTの実現性の確認 (2/3)




```
0000  ff ff ff ff ff ff a4 23  05 00 00 00 08 00 45 00   ·······# ·····E·
0010  01 b0 00 00 00 00 ff 11  00 00 0a c0 13 b4 7f 00   ···· ···· ····
0020  00 01 00 00 d4 31 01 9c  00 00 00 20 00 00 01 00   ·····1·· ··· ····
0030  00 00 3c 6c 84 69 00 00  00 00 02 05 00 00 00 00   ··<l·i·· ········
0040  02 26 08 00 45 04 01 7a  29 07 40 00 3f 11 f8 65   ·&··E··z )·@·?··e
0050  0a 00 02 01 0a 00 03 02  d0 4e 13 89 01 66 1a 16   ···· ···· ·N···f··
0060  01 00 19 00 00 07 40 03  fd 00 00 00 00 00 00 cd   ····· ·@· ····· ···
0070  00 00 00 05 00 00 00 0a  00 00 00 00 3c 6c 84 69   ···· ···· ····<l·i
0080  3c 6c 84 73 00 00 00 00  00 00 00 e2 00 01 00 02   <l·s···· ····· ····
0090  00 00 00 19 00 00 00 00  3c 63 f7 c7 3c 63 ···· ····<c··<c··
00a0  00 00 00 00 00 00 00 cc  00 03 00 01 00 00 ···· ···· ····
00b0  00 00 00 00 3c 73 41 25  3c 73 41 2c 00 00 ···· <sA% <sA, ····
00c0  11 13 89 00 00 29 e9 c8  5b c4 11 a9 00 0c ····· ···· [··· ····
00d0  00 00 00 00 00 00 00 01  00 00 13 89 00 00 ···· ···· ···· ····
00e0  00 10 00 00 ff f0 bd c0  36 37 38 39 30 31 ···· ···· 6789 01
00f0  34 35 36 37 38 39 30 31  32 33 34 35 36 37 4567 8901 2345 67
0100  30 31 32 33 34 35 36 37  38 39 30 31 32 33 0123 4567 8901 23
0110  36 37 38 39 30 31 32 33  34 35 36 37 38 39 6789 0123 4567 89
0120  32 33 34 35 36 37 38 39  30 31 32 33 34 35 2345 6789 0123 45
```

Eth + IP + UDP ヘッダ

INT メタデータ

Switch #3 INT META DATA

Switch #2 INT META DATA

Switch #1 INT META DATA

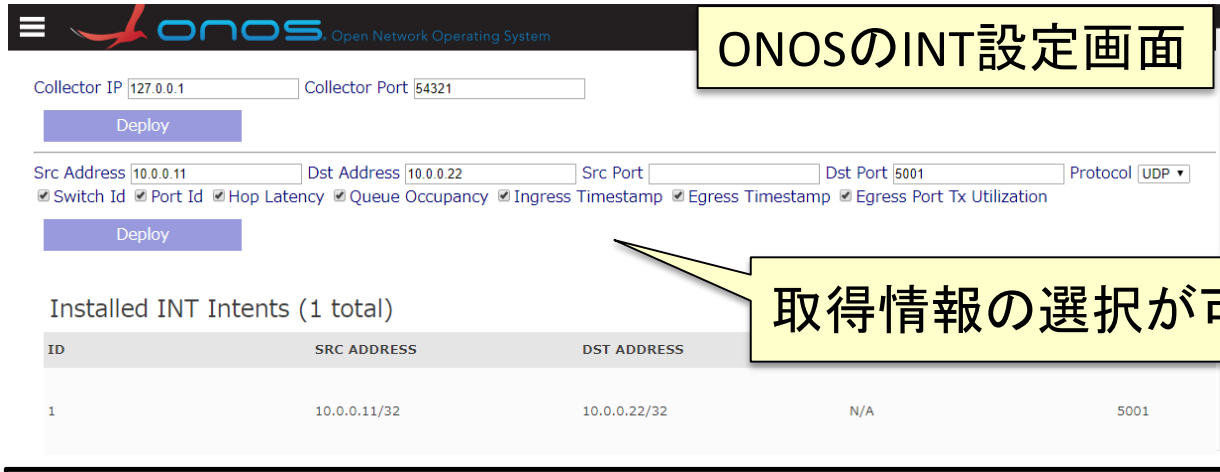
ペイロード

```
Frame 1: 488 bytes on wire (3904 bits), 488 bytes captured (3904 bits)
Ethernet II, Src: OpenNetw_00:00:00 (a4:23:05:00:00:00), Dst: Broadcast (ff:f
Internet Protocol Version 4, Src: 10.192.19.180, Dst: 127.0.0.1
User Datagram Protocol, Src Port: 0, Dst Port: 54321
INT-Report Protocol Data
  > Telemetry Report Fixed Header
    Encapsulated Header
  > INT Shim Header for TCP/UDP
  > INT metadata Header
  v INT metadata
    > Switch metadata #3                                   Switch #3
    v Switch metadata #2                                   Switch #2
        Switch ID: 226
        Ingress port ID: 1
        Egress port ID: 2
        Hop Latency: 25
        0000 0000 .... .... .... .... .... .... = Queue ID: 0
        .... .... 0000 0000 0000 0000 0000 0000 = Queue Occupancy: 0
        Ingress Timestamp: 1013184455
        Egress Timestamp: 1013184480
        0000 0000 .... .... .... .... .... .... = Queue ID: 0
        .... .... 0000 0000 0000 0000 0000 0000 = Queue Congestion: 0
    v Switch metadata #1                                   Switch #1
        Switch ID: 204
        Ingress port ID: 3
        Egress port ID: 1
        Hop Latency: 7
        0000 0000 .... .... .... .... .... .... = Queue ID: 0
        .... .... 0000 0000 0000 0000 0000 0000 = Queue Occupancy: 0
        Ingress Timestamp: 1014186277
        Egress Timestamp: 1014186284
        0000 0000 .... .... .... .... .... .... = Queue ID: 0
        .... .... 0000 0000 0000 0000 0000 0000 = Queue Congestion: 0
  > INT Tail Header
```

Copyright©2018 NTT corp. All Rights Reserved.

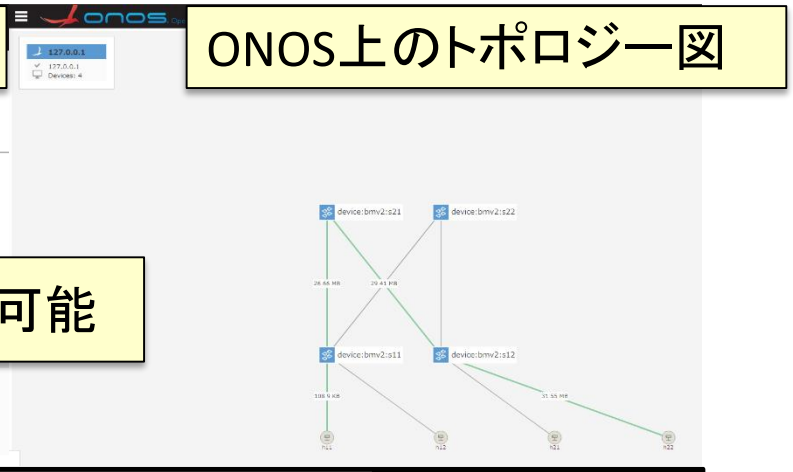
③ P4によるINTの実現性の確認 (3/3)

Innovative R&D by NTT



ONOSのINT設定画面

取得情報の選択が可能



ONOS上のトポロジー図



Grafana画面

Copyright©2018 NTT corp. All Rights Reserved.

P4検証を通して、わかったこと

- Stratumの実装が進んできている。
- P4プログラミングは簡単。
  - チュートリアルが整っている。
  - 数百行程度のコーディングで任意の機能がスイッチで実現できる。

- スイッチの環境構築に苦労
  - 検証時、ライブラリ不足などでビルドが通らなかった。
- 現在は、デバイス依存箇所が存在
  - ポート番号、リソース量等のデバイス依存要素を考慮しなければいけない場合あり
  - 検証時、BarefootスイッチはP4_14にのみ対応 (現在はP4_16にも対応)

Copyright©2018 NTT corp. All Rights Reserved.

今後の取り組み

Copyright©2018 NTT corp. All Rights Reserved.

PoC @NTT NS Lab.

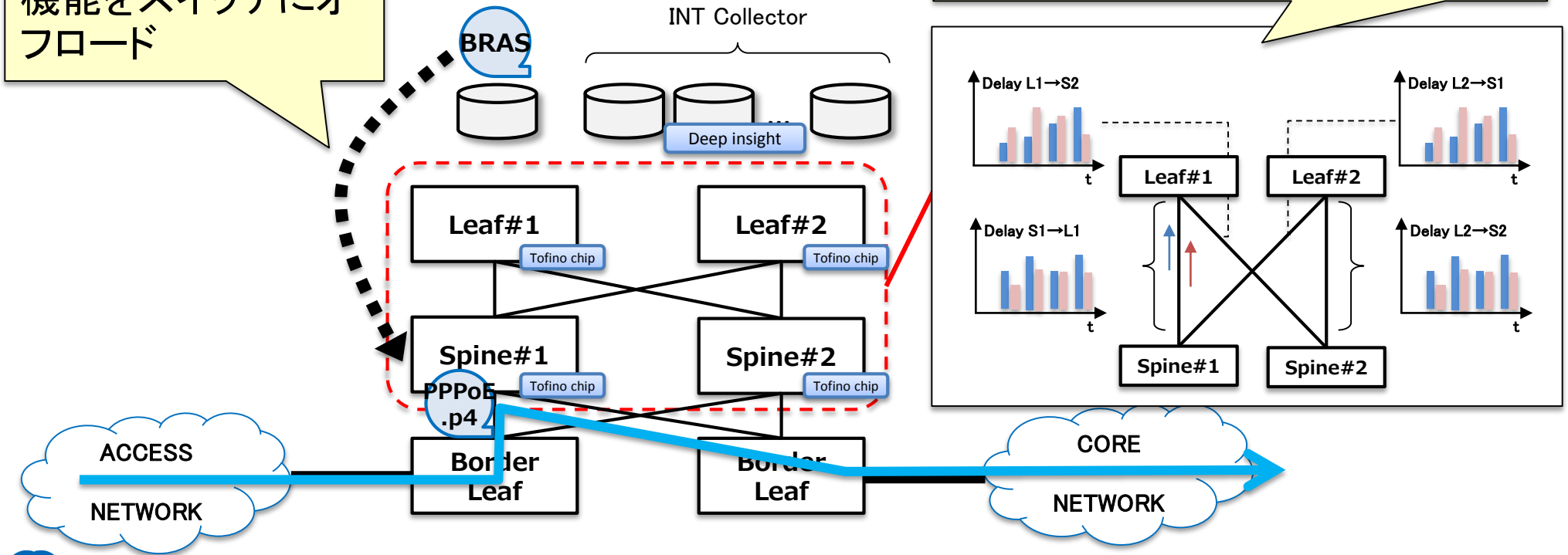
P4スイッチを用いたBNGのディスアグリゲーションのPoCを、NTTで実施予定(11月)

① Disaggregate PPPoE/L2TP Function with P4 program

② In-band-network telemetry with P4 program

PPPoE encap/decap機能をスイッチにオフロード

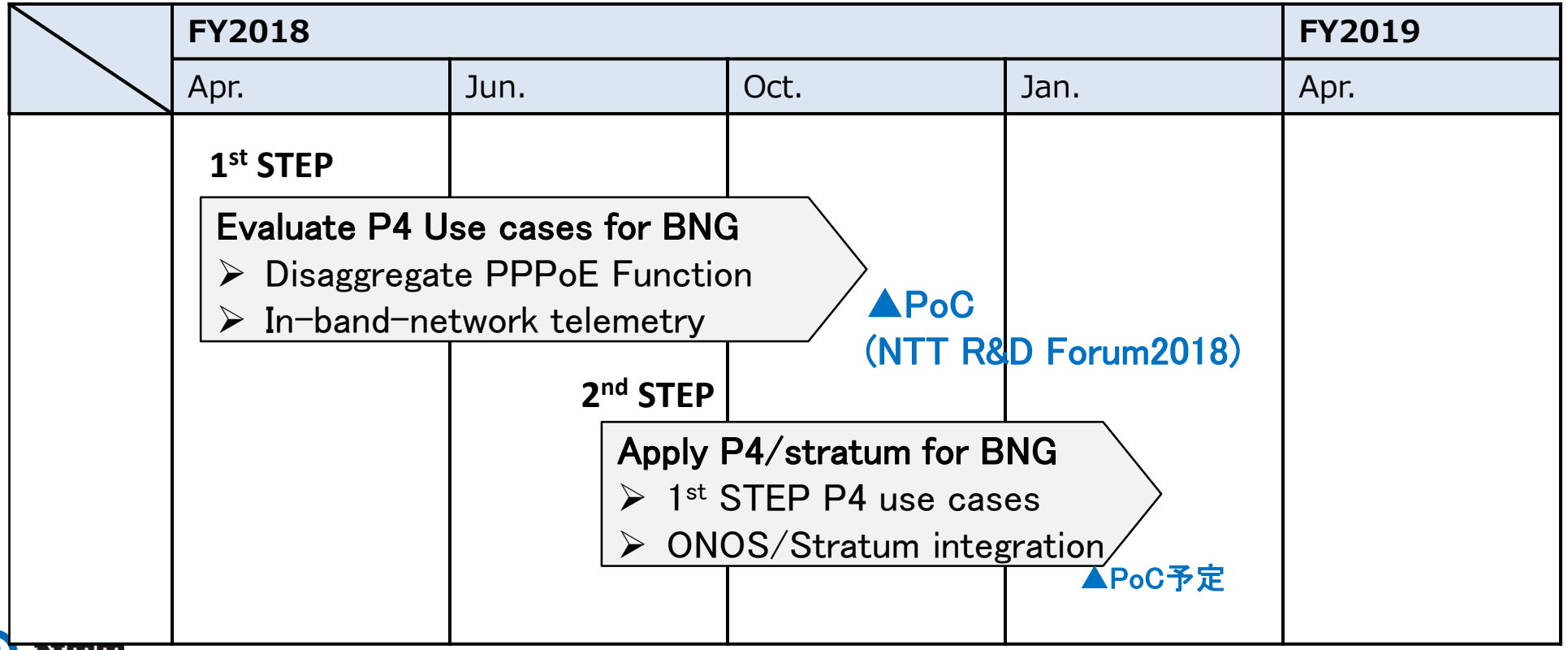
In-band-Network Telemetryによって各ネットワーク情報を可視化



Copyright©2018 NTT corp. All Rights Reserved.

今後のスケジュール

- P4/Stratum活用ユースケースの見極めを継続。
  - ✓ ２つのユースケースについて、NTT R&D Forum (2018.11)でPoC予定
  - ✓ 2nd STEPとして、ONOS/Stratum適応について検討。2019.1頃にPoC予定



Copyright©2018 NTT corp. All Rights Reserved.

まとめ

- P4/Stratumの実装が徐々に進んでいる。
- P4/Stratumを活用のユースケースとして、2つのアプローチを検討。
  ① VNFのオフロード、②通信品質の可視化
- P4を用いた機能実装は簡単である！

Copyright©2018 NTT corp. All Rights Reserved.